Self-supervised Representation Learning With Path Integral Clustering For Speaker Diarization

Prachi Singh, *Student Member, IEEE,* and Sriram Ganapathy, *Senior Member, IEEE*

***Abstract*—Automatic speaker diarization techniques typically involve a two-stage processing approach where audio segments of fixed duration are converted to vector representations in the first stage. This is followed by an unsupervised clustering of the representations in the second stage. In most of the prior approaches, these two stages are performed in an isolated manner with independent optimization steps. In this paper, we propose a representation learning and clustering algorithm that can be iteratively performed for improved speaker diarization. The representation learning is based on principles of self-supervised learning while the clustering algorithm is a graph structural method based on path integral clustering (PIC). The representation learning step uses the cluster targets from PIC and the clustering step is performed on embeddings learned from the self-supervised deep model. This iterative approach is referred to as self-supervised clustering (SSC). The diarization experiments are performed on CALLHOME and AMI meeting datasets. In these experiments, we show that the SSC algorithm improves significantly over the baseline system (relative improvements of 13% and 59% on CALLHOME and AMI datasets respectively in terms of diarization error rate (DER)). In addition, the DER results reported in this work improve over several other recent approaches for speaker diarization.**

***Index Terms*—Self-supervised learning, graph structural clustering, path integral clustering, speaker diarization.**

I. Introduction

SPEAKER diarization, the task of partitioning the input audio stream into segments based on speaker sources, has gained substantial interest in the recent years. The challenges in speaker diarization arise from short speaker turns, background noise, overlapping speech etc. Some of these challenges are highlighted in the recent DIHARD evaluations [1], [2], [3]. The early approaches to speaker diarization involved steps of change detection and gender classification [4]. The recent efforts have been directed towards a two stage approach of deriving embeddings from short segments followed by a clustering of the embeddings. The i-vector embedding extraction showed promising improvements for speaker diarization [5]. With the advancements in deep learning, the i-vector embeddings were successfully replaced with deep neural network (DNN) based embeddings like the x-vectors [6], which deploy a time delay neural network (TDNN). The x-vectors are also effective on short recordings [7].

In terms of clustering, the bottom-up clustering has been the most promising for speaker diarization [8]. The approach aims at successively merging until a stopping criterion is met [9]. The agglomerative hierarchical clustering (AHC) [10], a bottom-up approach, is the most popular method used for clustering. The mean-shift algorithm [11] is another approach which iteratively clusters by assigning data points to the means in a local vicinity. There has been efforts on pre-processing methods applied on embeddings like length normalization [12], recording level principal component analysis (PCA) [13] as well as the use of probabilistic linear discriminant analysis (PLDA) based affinity matrix computation for AHC [5], [14]. Viñals et al. [15] introduced session specific adaptation of PLDA using unsupervised clustering labels to re-estimate model parameters and speaker labels. The other approaches for clustering include spectral clustering approach [16] and the information bottleneck based approach [17]. Some of the recent approaches to speaker diarization have explored clustering free methods [18] or end-to-end models with two speaker conversations [19].

In this paper, we propose an approach based on self-supervised representation learning for speaker diarization. Self-supervised learning is a branch of unsupervised learning where the data itself provides supervision labels for model learning [20]. Self-supervised learning has been attempted for phoneme and speech recognition tasks [21]. For speaker diarization, the proposed approach involves iteratively updating embedding representations and cluster identities. The approach alternates between merging the clusters for a fixed embedding representation and learning the representations using the given cluster labels. We show that the proposed approach can be applied to traditional AHC and to graph based path integral clustering (PIC) [22].

This paper extends the previous work [23] using path integral clustering (PIC) and for the tasks with unknown number of speakers. We refer to the proposed approach of joint representation learning and clustering as self-supervised clustering (SSC). The key contributions from this work are:

- A simple method to jointly learn deep representations and speaker clusters from an unlabeled recording.
- Using graph based PIC for computing affinity measures between clusters.
- A stopping criterion based on eigen-values of the affinity matrix.
- Incorporating a temporal continuity criterion in the SSC to improve the smoothness of the clustering decisions.

The representation learning framework in the proposed SSC approach uses a triplet similarity based learning. The diarization experiments are performed on the CALLHOME

This work was supported by the grants from the British Telecom Research Center (BTRC) and with grants from the Department of Atomic Energy project 34/20/12/2018-BRNS/3408.

P. Singh and S. Ganapathy are with the Learning and Extraction of Acoustic Patterns (LEAP) lab, Department of Electrical Engineering, Indian Institute of Science, Bangalore, India, 560012. e-mail: {prachis, sriramg}@iisc.ac.in

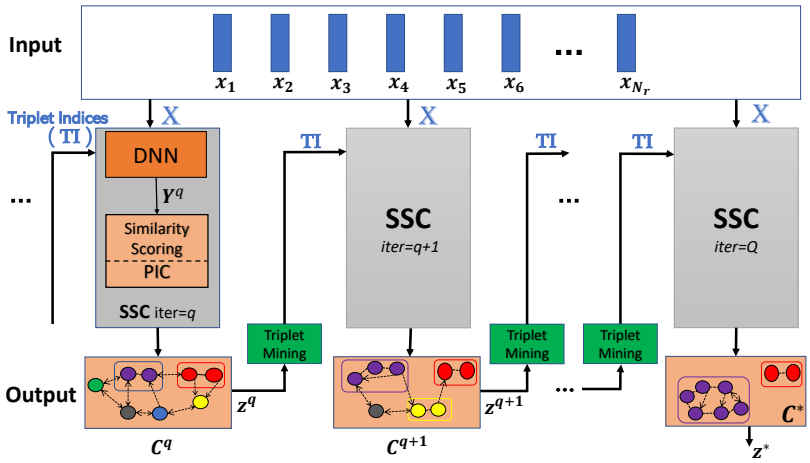

Fig. 1: Block schematic of the self-supervised diarization (SSC). The DNN embeddings are used in the path integral clustering (PIC). The clustering outputs from PIC create labels z^q used in sampling triplets for iteration $(q+1)$. The bottom row - circles represent embeddings, colors represent clusters and the dashed lines correspond to graph edges in PIC.

(CH) [24] and Augmented Multi-party Interactions (AMI) [25] datasets. Using the proposed SSC approach, we illustrate significant improvements over the x-vector AHC baseline system. The proposed approach also shows advancements over other published results on these datasets. Furthermore, the proposed approach can be used as an initialization for frame-level refinement based on variational Bayes (VB) hidden Markov model (HMM) [26], [27].

The rest of the paper is organized as follows. Section II gives an account of related prior work. Section III details the proposed SSC clustering. The dataset details are given in Section IV. The experiments on CALLHOME (CH) dataset and AMI dataset are reported in Section IV-C and Section IV-D respectively. This is followed by an analysis in Section V. The paper concludes with a summary in Section VI.

II. Related Work

Early works on diarization used short-term spectral feature vectors and unsupervised clustering based on Gaussian mixture models (GMMs) [28] or hidden Markov models (HMMs) [29]. In the last decade, embeddings based on dimensionality reduction of GMM statistics, termed as i-vectors [30], have been explored for speaker diarization tasks [5]. The i-vector embeddings are extracted from relatively long (1-2 sec.) segments of audio and they are clustered using Agglomerative Hierarchical Clustering (AHC) [10]. The affinity measure used initially was the cosine similarity score [31]. The model based affinity scoring using probabilistic linear discriminant analysis (PLDA) [32], which was successful in speaker verification [33], has also been incorporated for speaker diarization with AHC [5]. In the recent years, x-vector embeddings [6] trained using time delay neural networks (TDNNs) have replaced i-vectors for PLDA based AHC [34]. The use of recurrent neural networks (RNN) has also been explored for embedding extraction [35], [36].

In terms of the back-end modeling for speaker diarization, in addition to AHC, k-means clustering [37], and spectral clustering have also been explored [35]. The LSTM affinity measure has also been proposed for speaker diarization [38]. A fully supervised speaker diarization system has been investigated using unbounded interleaved RNN by Zhang et. al. [18]. The model is trained on conversational data directly with speaker labels. Separately, the application of end-to-end modeling for two speaker conversational data has been explored in [19]. In the end-to-end learning, the input features are fed to a model where the loss is either permutation-invariant cross entropy [39], [40] or clustering based [41]. Further to refine the boundaries of segmentation output in speaker diarization, a second re-segmentation step involving frame-level (20-30ms) modeling [26], [27] can be performed. This model the variational-Bayes HMM model which is typically initialized using a segment level diarization output [42].

The loss functions in clustering of images and text documents have explored cluster assignment losses such as k-means loss [43], spectral clustering loss [44] and agglomerative clustering loss [45]. These losses are derived from the unlabeled data itself and hence are self-supervised.

In this paper, we propose a novel approach for representation learning and clustering based on self-supervised learning. This approach, termed self-supervised clustering (SSC), is inspired by Yang et. al [45], which explores representation learning using agglomerative clustering based loss. The clustering is a forward operation while the representation learning is a backward operation. The motivation for this framework comes from the intuition that succinct speaker representations are beneficial for clustering while clustering results can provide self-supervisory targets for representation learning. The agglomerative clustering in the proposed SSC approach is based on path integral clustering (PIC), a graph-structural algorithm [22].

III. Self-supervised Clustering

The block schematic of the self-supervised clustering (SSC) algorithm is given in Figure 1. The inputs to the model are x-vector embeddings extracted from fixed length audio segments

(details of the x-vector embedding extraction are given in Section IV). The deep neural network (DNN) in Figure 1 is a two layer network. The output layer of the DNN generates representations that are used in the clustering.

The SSC implements iterative steps of clustering and representation learning. The clustering (forward operation) is performed using path integral clustering (described in Section III-C). The representation learning (backward operation) is performed using the DNN training with modified triplet similarity (described in Section III-D). The iterative process is repeated till the required number of clusters is reached or the stopping criterion is met (discussed in Section III-F).

A. Notation

The model parameters of the representation learning deep neural network (DNN) are denoted by $\boldsymbol{\theta}$. Let q denote the iteration index. Further, let

- $\mathbf{X}_r = \{\boldsymbol{x}_1, ..., \boldsymbol{x}_{N_r}\} \in \mathbb{R}^D$ denote the sequence of N_r input x-vector features for the recording r.
- $\mathbf{z}^q = \{z_1^q, ..., z_{N_r}^q\}$ denote the cluster labels for this recording, where each element takes a discrete cluster index value.
- $\mathbf{Y}^q = \{\boldsymbol{y}_1^q, ..., \boldsymbol{y}_{N_r}^q\} \in \mathbb{R}^d$ denote the output representations from the DNN model.
- $\mathbf{C}^q = \{\mathcal{C}_1^q, ..., \mathcal{C}_{N^q}^q\}$ denote the cluster set where each cluster $\mathcal{C}_i^q = \{\boldsymbol{y}_k^q | z_k^q = i, \forall k \in \{1, .., N_r\}\}$.
- $\mathcal{A}$ denote the affinity measure between two clusters.
- $\mathbf{S} \in \mathbb{R}^{N_r X N_r}$ denote pairwise similarity score matrix obtained using pair-wise similarity $s(i,j)$ between representations at time index i and j.
- N^q denote the number of clusters at q-th iteration
- N^* denote the target number of clusters in the algorithm.
- Q denote total number of iterations in the algorithm.
- Hyper-parameters:
  K - Number of nearest-neighbors used in PIC (Equation (2))
  σ - scaling factor in the computation of path integral in PIC (Equation (3))
  α - weighting factor in DNN learning (Equation (7))
  ϕ^q - threshold to estimate N^q.
  β, n_b - positive decaying factor and floor value respectively (Equation (8))

B. Background - Agglomerative Clustering

The AHC operates directly on short-segment embeddings (for the baseline system, these are the input x-vectors $\boldsymbol{X}_r$) and the algorithm merges two clusters at each step. The clusters to be merged at the q-th iteration are determined using,

$$\{\mathcal{C}_a, \mathcal{C}_b\} = \underset{C_i^q, C_j^q \in \mathbf{C}^q, i \neq j}{\operatorname{argmax}} \mathcal{A}\left(\mathcal{C}_i^q, \mathcal{C}_j^q\right) \tag{1}$$

The selected clusters $\{\mathcal{C}_a, \mathcal{C}_b\}$ are merged to form a new cluster $\{\mathcal{C}_a \cup \mathcal{C}_b\}^{q+1}$ for the next iteration. The merging process is stopped when the stopping criteria is met. The affinity $\mathcal{A}$ between clusters can be defined in multiple ways. It is computed using pairwise similarity between embeddings present in the similarity score matrix $\mathbf{S}$. The similarity score $(s(i,j))$ in state-of-the-art diarization systems use the PLDA modeling [34], while in the proposed SSC algorithm we use the cosine similarity measure. In conventional AHC, the affinity between two clusters can be defined using different linkage choices [46].

$\mathcal{A}\left(\mathcal{C}_a, \mathcal{C}_b\right)$ can be defined using:

- Single-linkage (Nearest-neighbor): The highest similarity score between two elements (one element from each cluster). $max_{i,j}\{s(i,j) : \boldsymbol{x}_i \in \mathcal{C}_a, \boldsymbol{x}_j \in \mathcal{C}_b\}$
- Complete-linkage (Farthest-neighbor): The lowest similarity score between two elements (one element from each cluster). $min_{i,j}\{s(i,j) : \boldsymbol{x}_i \in \mathcal{C}_a, \boldsymbol{x}_j \in \mathcal{C}_b\}$
- Average-linkage : The average similarity score of all pairs of elements (one from each cluster). $mean_{i,j}\{s(i,j) : \boldsymbol{x}_i \in \mathcal{C}_a, \boldsymbol{x}_j \in \mathcal{C}_b\}$

In our baseline experiments with AHC, we have used average linkage as the cluster affinity measure in Equation (1). The use of pairwise similarities for affinity may fail to capture the global structure of data [47]. This can be addressed in the graph based agglomerative clustering algorithms.

C. Path Integral Clustering

Path integral clustering (PIC) [22] is a graph-structural agglomerative clustering algorithm [47] where the graph encodes the structure of the embedding space. It exploits the connectivity of directed graph to efficiently cluster the embeddings. The PIC has been attempted for image segmentation and document clustering [48].

The PIC uses path integral as a structural descriptor of clusters. The clustering process is treated as a dynamical system, with vertices as input features (states of the system) and edge weights as transition probabilities between states. The affinity between two clusters is defined as the incremental path integral when the two clusters are merged. This affinity is based on the assumption that, if two clusters are closely connected, the stability of system can be enhanced by merging the two clusters.

The PIC algorithm consists of the following steps,

- Defining the digraph: We build the directed graph based on similarity scores between embeddings. The digraph is denoted as $G = (V, E)$, where V is the set of vertices corresponding to the embeddings in $\boldsymbol{Y}$ and E is the set of edges connecting the vertices. The adjacency matrix $\boldsymbol{W} \in \mathbb{R}^{N_r X N_r}$ is a sparse matrix defined as,

$$[W]_{ij} = \begin{cases} \frac{1}{1+exp(-s(i,j))}, & \text{if } \boldsymbol{y}_j \epsilon N_i^K. \\ 0, & \text{otherwise.} \end{cases} \tag{2}$$

  where, N_i^K is the set of K-nearest neighbors of $\boldsymbol{y}_i$. The transition probability matrix $\boldsymbol{P}$ is obtained from the adjacency matrix $\boldsymbol{W}$ by normalizing each row with its sum. The transition probability matrix $\boldsymbol{P}$ contains the edge weights of the graph. The directed edge connecting node i to node j exists only if the value of $[W]_{ij}$ is non-zero.
- Cluster initialization: We group the embeddings $\boldsymbol{Y}$ with their first nearest neighbour to form $N_r/2$ clusters. If the clusters have common elements, these are further merged.
- Defining path integral: We define path integral $\mathcal{S}_{\mathcal{C}_a}$ of a cluster $\mathcal{C}_a$ as the weighted sum of probabilities of all

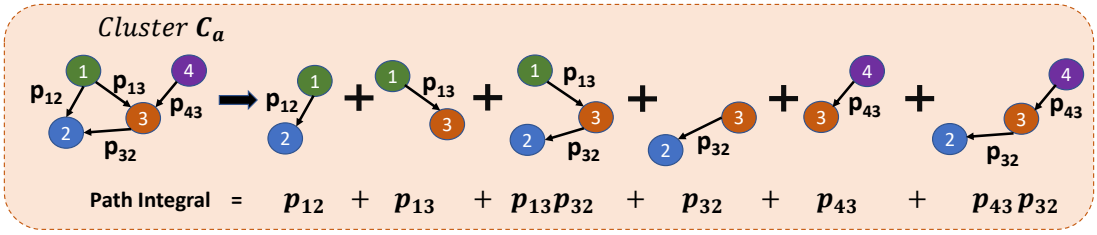

Fig. 2: Computation of path integral of a graph. Cluster C_a is represented as graph with edge weights as transition probabilities obtained using Equation (2). Path integral is the sum of probabilities of all possible paths in the graph.

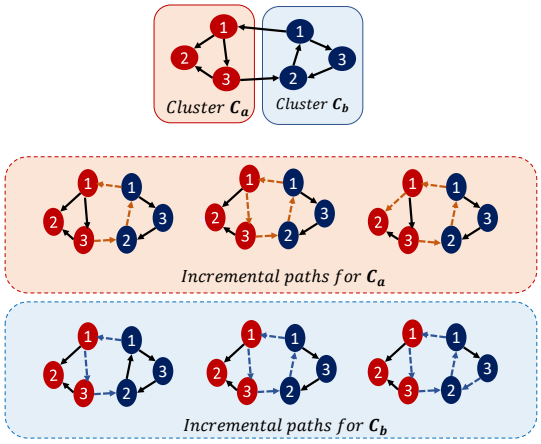

Fig. 3: Illustration of incremental paths of clusters C_a and C_b for computing affinity using Equation (6). Second and third rows show paths for $C_a \cup C_b$. Second row highlights paths which start and end in cluster C_a with purple dashed arrows. Third row shows the paths which start and end in C_b with blue dashed arrows.

possible paths from any vertex i to any other vertex j, where i, j vertices belong to the cluster $\mathcal{C}_a$ and all the vertices along the path also belong to cluster $\mathcal{C}_a$. An illustration of path integral is shown in Figure 2. The unnormalized pairwise path integral[1] g_{ij} over all paths (of lengths k ranging from 1 to ∞) from vertex i to j for $\mathcal{G}_{\mathcal{C}_a}$ is given as:

$$g_{ij} = \delta_{ij} + \sum_{k=1}^{\infty} \sigma^k \sum_{\gamma \in \Gamma_{ij}^{(k)}} \prod_{s=1}^{k} p_{u_{s-1},u_s} \tag{3}$$

where, $u_0 = i$, $u_k = j$, p_{u_{s-1},u_s} is transition probability from vertex u_{s-1} to u_s and δ_{ij} is Kronecker delta function defined as $\delta_{ij} = 1$ if $i = j$ and 0 otherwise. $\Gamma_{ij}^{(k)}$ is the set of all possible paths from i to j of length k. The constant $0 < \sigma < 1$ gives more weight to shorter paths compared to longer paths. The path integral $\mathcal{S}_{\mathcal{C}_a}$ of the cluster $\mathcal{C}_a$ is then the summation of g_{ij} over all pairs of vertices $i, j \in \mathcal{C}_a$ normalized by $|\mathcal{C}_a|^2$.

[1]The proof of the path integral equation is given in Appendix-A

We define the conditional path integral $\mathcal{S}_{\mathcal{C}_a|\mathcal{C}_a\cup\mathcal{C}_b}$ as the path integral of all paths in $\mathcal{C}_a \cup \mathcal{C}_b$ such that the paths start and end with vertices belonging to $\mathcal{C}_a$. As shown in Appendix VI, the normalized path integral and the normalized conditional path integral converges to

$$\mathcal{S}_{\mathcal{C}_a} = \frac{1}{|\mathcal{C}_a|^2} \mathbf{1}^T \left(\boldsymbol{I} - \sigma \boldsymbol{P}_{\mathcal{C}_a}\right)^{-1} \mathbf{1} \tag{4}$$

$$\mathcal{S}_{\mathcal{C}_a|\mathcal{C}_a\cup\mathcal{C}_b} = \frac{1}{|\mathcal{C}_a|^2} \mathbf{1}_{\mathcal{C}_a}^T \left(\boldsymbol{I} - \sigma \boldsymbol{P}_{\mathcal{C}_a\cup\mathcal{C}_b}\right)^{-1} \mathbf{1}_{\mathcal{C}_a} \tag{5}$$

where, $\boldsymbol{P}_{\mathcal{C}_a}$ and $\boldsymbol{P}_{\mathcal{C}_a\cup\mathcal{C}_b}$ are the sub-matrices of the transition probability matrix $\boldsymbol{P}$ obtained by selecting vertices that belong to $\mathcal{C}_a$ and $\mathcal{C}_a \cup \mathcal{C}_b$ respectively. Here, $|\mathcal{C}_a|$ denotes the cardinality (# of vertices) of cluster $\mathcal{C}_a$, $\mathbf{1}$ is a column vector of all ones of size $|\mathcal{C}_a|$ and $\mathbf{1}_{\mathcal{C}_a}$ is a binary column vector of size $|\mathcal{C}_a \cup \mathcal{C}_b|$ with ones at all locations corresponding to the vertices of $\mathcal{C}_a$ and zeros at all locations corresponding to the vertices of $\mathcal{C}_b$. Note that the identity matrix $\boldsymbol{I}$ used in Equation (4) and (5) are of dimensions $|\mathcal{C}_a|$ and $|\mathcal{C}_a \cup \mathcal{C}_b|$ respectively.

- Cluster merging: The cluster affinity measure for the PIC algorithm is computed as,

$$\mathcal{A}\left(\mathcal{C}_a, \mathcal{C}_b\right) = \left[\mathcal{S}_{\mathcal{C}_a|\mathcal{C}_a\cup\mathcal{C}_b} - \mathcal{S}_{\mathcal{C}_a}\right] + \left[\mathcal{S}_{\mathcal{C}_b|\mathcal{C}_a\cup\mathcal{C}_b} - \mathcal{S}_{\mathcal{C}_b}\right] \tag{6}$$

where the term inside each square bracket is called incremental path integral. The computation of the incremental path integrals is illustrated in Figure 3. A higher value of affinity between two clusters indicates the presence of more dense connections between them. Thus, merging the clusters with maximum affinity will form a more coherent cluster. Using the definition of affinity (Equation (6)), the clusters with maximum affinity are merged at each iteration as shown in Equation (1).

D. DNN Training - Dynamic Triplet Similarity

The clustering algorithm generates labels $\boldsymbol{z}^{q-1}$ where each z_i^{q-1} can take discrete values from $\{1, .., N_{q-1}\}$, with N_{q-1} being the number of clusters at the end of iteration $(q-1)$. We use the dynamic triplet similarity to train the DNN model. The term dynamic is used to indicate the formation of new triplets after each iteration. We form a positive pair $\{\mathbf{y}_i^{q-1}, \mathbf{y}_j^{q-1}\}$ by selecting the anchor $\mathbf{y}_i^{q-1}$ and positive sample $\mathbf{y}_j^{q-1}$ from each cluster $\mathcal{C}_a^{q-1}$. Here, $\mathbf{y}^{q-1}$ denotes the representations formed at the DNN output using $\boldsymbol{\theta}^{q-1}$. The negative pair is formed by

Algorithm 1: SSC algorithm for joint representation learning and clustering

Initialize:$(q = 0)$
$\boldsymbol{\theta}^0 \leftarrow$ (Whitening+PCA)
$\boldsymbol{Y}^0 \leftarrow$ PCA outputs
If unknown $N^* \rightarrow N^* = 1$
while *continue* **do**
1) $q = q + 1$
2) Sample triplets based on $\boldsymbol{z}^{q-1}$
3) $\boldsymbol{\theta}^{q-1} \xrightarrow{\text{DNN with triplet training}} \boldsymbol{\theta}^q$
4) $\boldsymbol{X}_r \xrightarrow{\text{Forward Pass}(\boldsymbol{\theta}^q)} \boldsymbol{Y}^q$
5) $\tilde{N}^q =$Estimate_$N^q(\boldsymbol{Y}^q, \phi^q, N^{q-1})$
6) $N^q = max(N^*, \tilde{N}^q)$
7) **if** $N^q == N^*$ *or* $q == Q$ **then**
$Q = q$
$N^* = N^Q$
break
end
8) $\boldsymbol{Y}^q \xrightarrow{\text{PIC}(N^q \text{ clusters})} \boldsymbol{z}^q = \{z_1^q, ..., z_{N_r}^q\}$
end
Termination:
$\boldsymbol{X}_r \xrightarrow{\text{DNN training + Forward Pass}(\boldsymbol{\theta}^*)} \boldsymbol{Y}^* = \{\boldsymbol{y}_1^*, ..., \boldsymbol{y}_{N_r}^*\}$
$\{\boldsymbol{y}_1^*, ..., \boldsymbol{y}_{N_r}^*\} \xrightarrow{\text{PIC}(N^* \text{ clusters})} \{z_1^*, ..., z_{N_r}^*\}$

randomly sampling the embedding $\mathbf{y}_l^{q-1}$ from any other cluster $\mathcal{C}_b^{q-1}$ $(a \neq b)$. The DNN model parameters $\boldsymbol{\theta}$ are updated based on the following optimization criteria.

$$\boldsymbol{\theta}^q = \underset{\boldsymbol{\theta}}{\operatorname{argmax}} \sum_{i,j,l} \left[s(i,j) - \alpha \left(s(i,l) + s(j,l) \right) \right] \tag{7}$$

where, $s(i,j)$ is pairwise similarity score between $\boldsymbol{y}_i^{q-1}$ and $\boldsymbol{y}_j^{q-1}$. Here, $0 < \alpha \leq 1$ is a weighting factor which controls the discriminability of the negative pairs. In the triplet formation, we also try to uniformly sample similar number of anchors from all the clusters by suitably oversampling the negative pairs for the same positive pair.

E. SSC Algorithm Overview

The overview of the SSC[2] algorithm is given in Algorithm 1. The DNN model is trained using gradient ascent with triplet similarity. At the q-th iteration, positive and negative triplet pairs are formed using the cluster label indices $\mathbf{z}^{q-1}$ of the previous iteration. We calculate the triplet similarity using these triplets and learn the DNN model parameters $\boldsymbol{\theta}^q$ with the inputs as x-vector features $\boldsymbol{X}_r$.

Once the DNN model is trained, we obtain embeddings $\boldsymbol{Y}^q$ from the last layer of DNN. We compute cosine similarity scores matrix $\boldsymbol{S}$ using the embeddings $\boldsymbol{Y}^q$ for the recording.

The PIC based clustering is performed using the similarity measure such that $N^q \leq N^{q-1}$. We estimate N^q based on the stopping threshold ϕ^q discussed in next section. We start with N_r clusters and perform multiple merges using the criterion

[2]Our implementation of SSC-PIC is available at https://github.com/iiscleap/SSC.git

Algorithm 2: Estimate number of clusters N^q: Estimate_$N^q(\boldsymbol{Y}^q, \phi^q, N^{q-1})$

1) $\boldsymbol{Y}^q \xrightarrow{\text{PIC}(N^{q-1} \text{ clusters})} \boldsymbol{A} \in \mathbb{R}^{N^{q-1} X N^{q-1}}$: Affinity matrix of N^{q-1} clusters such that $[A]_{ij} = \mathcal{A}(\mathcal{C}_i, \mathcal{C}_j)$
2) $\lambda_1 \geq \lambda_2 \geq ... \geq \lambda_{N^{q-1}}$: Eigen values of $\boldsymbol{A}$; $total\_energy = \sum_{i=1}^{N^{q-1}} \lambda_i$
3) $\boldsymbol{v} \in \mathbb{R}^{N^{q-1}}$: cumulative explained variance ratio; $[v]_k = \frac{\sum_{i=1}^{k} \lambda_j}{total\_energy}$ where $k \in \{1, ..., N^{q-1}\}$
4) $N^q = \underset{k}{\operatorname{argmax}}([v]_k \leq \phi^q)$
5) return N^q

defined in Equation (1) thereby reducing the total number of clusters in the q-th iteration. If $N^q = N^*$ or the stopping criterion is met, then the algorithm is stopped. We perform one more iteration of DNN training and clustering to generate the final diarization results. Else, the iteration index is updated and the steps described above are repeated for the $(q+1)$-th iteration. If N^* is unknown, then the stopping criteria is based on total number of iterations Q or till we reach $N^q = 1$ (whichever is achieved first).

F. Estimation of Number of Clusters (Algorithm 2)

The number of clusters N^q required for clustering at each iteration q is estimated based on explained variance. In this approach, we compute affinity matrix $\boldsymbol{A}$ of N^{q-1} clusters using Equation (6). The diagonal elements are set as the maximum value of non-diagonal elements of the matrix. Then, we compute eigen values of the matrix and arrange them in descending order. We accumulate the eigen values till the cumulative explained variance ratio, which is the ratio of accumulated eigen values to total sum of eigen values, reaches a stopping threshold ϕ^q for the q-th step. The number of accumulated eigen-values at this step is denoted as N^q.

G. Incorporating Temporal Continuity

In the given audio stream, the event that two neighboring x-vector embeddings come from the same speaker is more likely than the event that they belong to different speakers. This heuristic can be incorporated in the clustering algorithm. After DNN training, we obtain similarity score matrix $\boldsymbol{S}$ using the output embeddings and multiply the similarity score $s(i,j)$ with an exponential decaying function of the temporal distance between i and j.

$$s'(i,j) = s(i,j)\beta^{min(n_b, |i-j|)} \tag{8}$$

where, β is a positive decaying factor $(0 < \beta < 1)$, $|i-j|$ is the absolute value of the time difference between segment i and j, n_b denotes a floor value which prevents the similarity score from going too low for segments that are farther in time. The above modification of similarity score encourages neighboring clusters to merge and have smooth transition among clusters.

TABLE I: The x-vector training configurations for baseline of CH and AMI datasets.

Parameter	CH	AMI
Sampling Rate	8kHz	16kHz
Train set	SWBD, SRE 04-08	Voxceleb 1,2
# speakers	4,285	7,323
Input features	23D MFCCs	30D MFCCs
X-vector dimension	128	512

IV. Experimental Setup

A. Evaluation Data

- CALLHOME: The CALLHOME (CH) dataset [24] is a collection of multi-lingual telephone data sampled at 8kHz, containing 500 recordings, where the duration of each recording ranges from 2-5 mins. The number of speakers in each recording varies from 2 to 7, with majority of the files having 2 speakers. The CH dataset is divided equally into 2 different sets, CH1 and CH2, with similar distribution of number of speakers.
- AMI: The AMI dataset [25] comprises of meetings recorded at different sites (Edinburgh, Idiap, TNO, Brno) with sampling frequency of 16kHz. We use the dev and eval sets of the single distant microphone (SDM) condition from the official speech recognition partition of AMI dataset. The AMI-dev set contains 18 meeting recordings and AMI-eval set contains 16 meeting recordings. All the meeting recordings contain 4 speakers except one recording from the eval set which has 3 speakers. The duration of each recording ranges from 20-60 mins.

B. X-vector Extraction

The details of the x-vector model are given in Table I. The TDNN model consists of 5 layers of time-delay neural network and two layers of feed forward architecture operating at frame-level followed by a pooling layer which generates mean and standard deviation statistics at the segment level [6]. The segment level statistics are passed through two feed forward layers to the target layer. The model is trained using cross-entropy loss on the training speakers using the asynchronous stochastic gradient descent (ASGD) algorithm. The first affine layer (before the non-linearity) after the pooling layer is used as the x-vector feature. Table I shows the differences in x-vector training configuration for CH and AMI datasets.

C. Experiments on CH Dataset

1) Baseline: The baseline system used for the CH diarization task comes from the Kaldi recipe[3]. It involves the extraction of 23 dimensional mel-frequency cepstral coefficients (MFCCs). A sliding window mean normalization is applied over a 3s window after the feature extraction. For training the x-vector TDNN model and the PLDA model, we use the SRE 04-08 and Switchboard cellular datasets as given in the Kaldi recipe [49]. This training set had $4,285$ speakers. For the diarization task, speech segments (1.5s chunks with 0.75s overlap) are converted to 128 dimensional neural embeddings (x-vectors) [6]. The x-vectors are processed by applying whitening transform obtained from the held-out set followed by length normalization [12]. An utterance level PCA is applied for dimensionality reduction [13] preserving 10% of total energy. These embeddings are used in the PLDA model to compute the similarity score matrix. The clustering is performed using the AHC algorithm as discussed in Section III-B. The AHC stopping criterion is determined using held-out data (CH1 is used for diarization on CH2 data and vice-versa).

2) SSC Algorithm Implementation: We use the same setup in the baseline system to extract the x-vector features. The number of x-vectors per recording (N_r) range from 50-700. The DNN model is a two-layer fully connected DNN. The first layer has 128 input and hidden nodes. The second layer has 10 output nodes. The first layer of the DNN also realizes the unit length normalization as the non-linearity. The learning rate is set at 0.001. The model is trained using the triplet similarity defined in Equation (7). We do not split the training triplet samples into mini-batches, but rather perform a full batch training with Adam optimizer [50]. We use a parameter $\eta = 0.5$ (defined as the fraction of the training similarity measure at the zeroth iteration compared to the current iteration) for early-stopping of the DNN model training. Typically, this model training involves 5-10 epochs. The DNN outputs (10 dimensional embeddings) are used to compute the pairwise similarity score matrix using cosine similarity for the PIC.

D. Experiments on AMI Dataset

1) Baseline: The baseline x-vector system uses the DIHARD Challenge recipe[4] for the AMI dataset. It involves the extraction of 30 dimensional MFCCs. The x-vectors are of dimension 512. For training the x-vector extractor and the PLDA model in AMI experiments, we trained the TDNN model using 16 kHz data from VoxCeleb-1 [51] and VoxCeleb-2 [52] datasets containing $7,323$ speakers. In the post-processing of the x-vectors, we use the held-out dataset from the second DIHARD challenge [2] for computing the whitening transform. The rest of the steps in the PLDA-AHC baseline system follow the same processing pipeline used in the CH baseline system.

2) SSC Algorithm Implementation: The DNN model is a two-layer feed forward architecture (similar to the CH dataset). The first layer has 512 input and hidden nodes. The second layer has 30 output nodes (similar to the PCA dimension used in the baseline system). In the AMI dataset, the number of x-vectors per recording range from 1000-4000. Hence, a large number of triplets can be generated for each recording. We resort to the use of minibatches in DNN model training. A validation split is also formed in each recording. The loss on the validation set is used to decide the stopping point for training. We perform a learning rate annealing [38] in the DNN model training. The initialization of the DNN model follows the strategy used in CH dataset.

[3] https://kaldi-asr.org/models/m6

[4] https://github.com/iiscleap/DIHARD_2019_baseline_alltracks

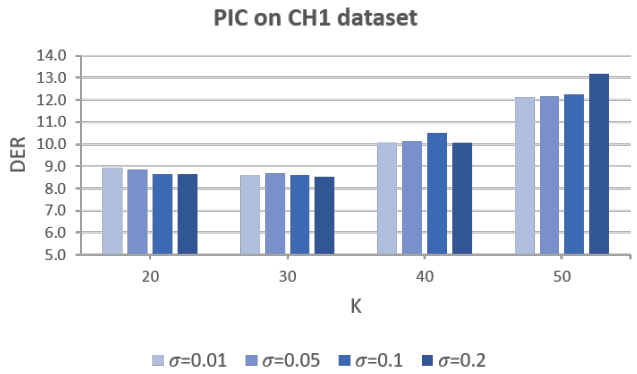

Fig. 4: DER of CH1 dataset for different choices of number of nearest-neighbors K and scaling factor σ.

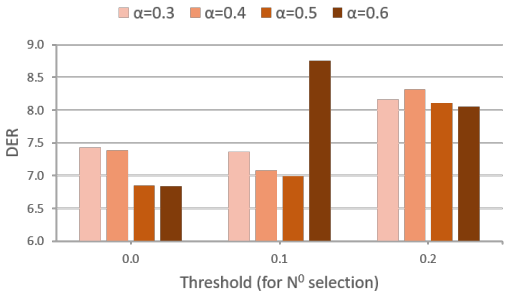

Fig. 5: Bar plot of DER vs threshold for N^0 selection on CH1 subset of CALLHOME for different choices of α parameter.

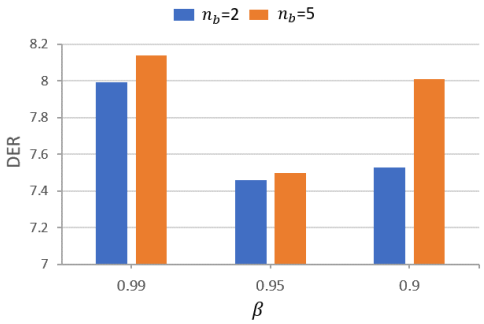

Fig. 6: Effect of temporal weighting (β, n_b) on DER for CH1 dataset. The best DER is obtained for $n_b = 2$ with $\beta = 0.95$.

V. Results and Analysis

The performance metric in all the experiments used in this paper is the diarization error rate (DER) computed with a 250 ms collar and ignores overlapping segments. For all our experiments, we use the oracle speech activity decisions.

A. Choice of Hyper-parameters

- **Number of nearest-neighbours K and scaling factor σ:** We choose K and σ values based on experiments on the CH1 subset of the CALLHOME dataset. Figure 4 shows the overall DER on CH1 for different values of K and σ. We observe that as we increase K, the DER is increasing. The higher K value also increases computational complexity. Based on the results, the value of K is chosen as 30. For a fixed K, we vary σ from 0.01 to 0.2. The parameter σ has only a minor impact on the final DER performance. The value $\sigma = 0.1$ is chosen for all our experiments.
- **Initial number of clusters (N^0) and α:** The DNN training is performed only after the PIC/AHC clustering algorithm is performed for a few iterations to reduce the number of clusters from N_r to N^0. If the value of N^0 is too large, then the discriminative triplet similarity measure tends to increase the within speaker diversity (controlled by factor α). However, if the value of N^0 is reduced significantly, then the clusters formed can be potentially impure in terms of the speaker identity of the elements in the cluster.

  For CH dataset, we initialize the model using AHC and vary the threshold applied on similarity scores to stop at N^0 number of clusters. In Figure 5, we observe that for lower threshold (smaller N^0), higher α (more discriminative) is better but as we increase threshold (larger N^0), lower α (less discriminative) is optimal. We obtain the best DER with threshold$= 0.0$ and $\alpha = 0.6$, which is used in CH experiments. In the AMI experiments, we choose N^0 as the smallest number obtained using the threshold ϕ for the explained variance on the affinity.
- **Temporal weighting factors β and n_b:** Figure 6 shows the effect of the hyper-parameters β and n_b used in incorporating temporal continuity (Equation 8). The best DER is obtained on the CH1 dataset using $n_b = 2$ and $\beta = 0.95$. The inclusion of temporal continuity discourages frequent speaker turns. Therefore, it is more useful for recordings which are longer in duration and for those with less frequent speaker turns. We found the best parameters on CH1 subset and used it for both the CH and AMI experiments.

B. Triplet sampling strategy

We explore different sampling strategies and measure DER performance on CH1 dataset:

1) **Hard Sampling**: We sample positive pairs from the same cluster and negative from the nearest neighbor of the anchor belonging to a different cluster. DER obtained on CH1 is 9.4%.
2) **Random Sampling**: Here, we sample positive pairs from the same cluster and randomly sample negative pairs from any other cluster. We get DER$= 6.8\%$.
3) **Easy Sampling**: In this case, we sample positive pairs from the same cluster and the negative pair using the farthest data point from the anchor that belongs to a different cluster. DER performance is 8.0%.

Although hard sampling is typically preferred in triplet mining on supervised data, we find that the random sampling works best in our case of unsupervised clusters. The hard sampling may potentially bias the model to disallow the merge of same speaker clusters. However in case of easy sampling, the SSC model learning is somewhat compromised as the negative samples do not provide a discriminative learning criterion.

C. SSC Initialization

The initialization of the DNN model uses the whitening transform and the recording level PCA from the baseline

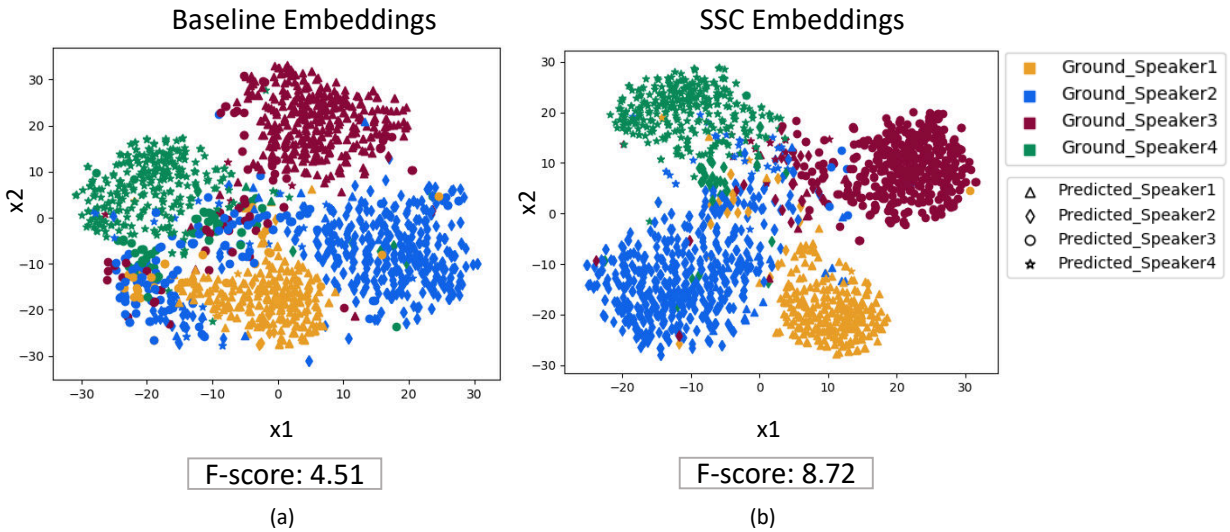

Fig. 7: t-SNE based visualization of embeddings extracted on 1.5s audio segments from the recording AMI-ES2011c-SDM. (a) the baseline x-vectors post-processed with whitening transformation and PCA, (b) embeddings obtained after SSC algorithm (DNN embeddings $\boldsymbol{Y}$). The colors represent speakers in the ground truth and the shapes represent predicted clusters.

system for the first and the second layer respectively. The output of the model are the embeddings $\boldsymbol{Y} = \{\boldsymbol{y}_1, .., \boldsymbol{y}_{N_r}\}$ which are used to perform clustering (AHC/PIC) till N^0 number of clusters. For CH, we use AHC clustering with threshold= 0.0 to generate N^0 cluster labels for each $\boldsymbol{y}_k$. For the AMI SSC-AHC training, we used PLDA scoring with learnable PLDA parameters. We use PLDA-AHC with threshold= 0.0 to generate initial speaker labels . For SSC-PIC, we perform PIC clustering using either $N^0 = N^*$ in the known N^* case or we generate N^0 using a stopping threshold $\phi = 0.7$ on the eigen value ratio (Algorithm 2) in the unknown N^* case.

D. Stopping Criterion

In the SSC training, the stopping criterion for the DNN representation learning model is based on the triplet-loss. We use the stopping threshold of 0.5 times the triplet-loss at the initial epoch of the model training. The stopping criterion for the SSC merging process is done using the eigen-value ratio in the affinity matrix (Sec. III-F). The exact threshold is selected based on the development data (similar to the AHC threshold estimation).

E. Visualization of Embeddings

Figure 7 shows the visualization of the embeddings from the baseline system (x-vec. + AHC) and the proposed SSC model for one recording from the AMI dataset. The visualization is performed using t-distributed stochastic neighborhood embedding (t-SNE) which performs an unsupervised dimensionality reduction of the embeddings [53]. The embeddings from the baseline system are shown in Figure 7(a) where the x-vectors are extracted from 1.5s segments of audio with half overlap followed by a whitening transform and PCA dimensionality reduction at the recording level. The embeddings from the SSC algorithm are shown in Figure 7(b) for the same recordings. The colors and shapes indicate the ground-truth and predicted speaker clusters respectively. The ideal embeddings would have one-to-one mapping between shapes and colors. As seen in this Figure, the SSC algorithm provides improved cluster separability (highlighted using F-score in the plots) along with improved association with the ground-truth speakers.

TABLE II: DER (%) for different systems when number of speakers (N^*) is known and unknown for the full CH dataset.

System	Known N*	Unknown N*
x-vec. + cosine + AHC	8.9	10.0
x-vec. + cosine + Spec. Clus.	9.4	11.9
x-vec. + PLDA + AHC (Baseline)	7.0	8.0
x-vec. + cosine + PIC	7.7	9.3
Self-supervised AHC (SSC-AHC) [23]	6.4	8.3
Self-supervised PIC (SSC-PIC)	6.4	7.5
+ Temporal continuity	**6.3**	**7.0**

F. CH Diarization Results

The diarization results on the CH dataset are shown in Table II. Here, the initial set of experiments use x-vector embeddings (with post-processing) clustered with different algorithms (AHC/PIC/Spectral Clustering) and similarity measures (cosine/PLDA). The spectral clustering is a graph partitioning algorithm which uses eigen values and eigen vectors of Laplacian matrix to perform clustering [54]. The Laplacian is computed from similarity score matrix. We use the eigen vectors with non-zero eigen values to perform k-means clustering.

The PLDA-AHC approach gives the best performance for x-vector embeddings and this is used as the baseline system for comparison with the proposed SSC algorithm. For known N^*, Self-Supervised PIC (SSC-PIC) shows marginal improvement compared to SSC-AHC. However, for unknown N^*, we obtain considerable improvements for the SSC-PIC algorithm. The best results improve significantly over the baseline system for

TABLE III: DER (%) for different systems when number of speakers (N^*) is known and unknown for the AMI dataset.

System	Known N^*		Unknown N^*	
	Dev.	Eval.	Dev.	Eval.
x-vec + cosine + AHC	34.6	30.2	18.2	15.5
x-vec + cosine + Spec. Clus.	30.2	25.5	40.0	31.1
x-vec + PLDA + AHC (Baseline)	15.7	16.0	13.7	16.3
SSC-PLDA-AHC	9.4	11.1	10.7	11.6
x-vec + PLDA + PIC	9.4	9.3	9.8	10.4
x-vec + cosine + PIC	8.9	7.3	9.0	7.3
SSC-PIC	7.3	7.2	8.1	7.6
+ Temporal continuity	**6.2**	**6.4**	**6.4**	**6.7**

both the cases of known and unknown number of speakers. The relative improvements in known N^* and unknown N^* respectively for the SSC algorithm over the baseline system are 10% and 13%. We also performed paired student-t test to check the statistical significance of file-level DER improvements obtained for the SSC-PIC over the baseline system in Table II. In this statistical t-test, comparing the baseline and SSC-PIC system, we find a t-statistic of $t = 2.2$ and p value of $p = 0.03$. This indicates that the improvements seen in the proposed approach are statistically significant. In order to show the improvement in per-cluster variance, we computed the F-ratio (ratio of between speaker variance to within speaker variance) based on the similarity scores. We found the F-ratio to improve by 130% on an average on the CH1 dataset with the SSC training.

G. AMI Diarization Results

The diarization results on the AMI dataset are shown in Table III. Here, the initial set of experiments report using x-vector embeddings (with post-processing) clustered with different algorithms (AHC/PIC/Spec. Clus.) and similarity measures (cosine/PLDA). As the PLDA-AHC performance is significantly better than cosine-AHC, we performed SSC-AHC with PLDA scoring. Our SSC-PLDA-AHC system outperforms the baseline for all scenarios. However, the PIC algorithm achieves the best results compared to other clustering results on AMI. PIC algorithm can be used with PLDA and cosine scoring but the performance is comparatively better with cosine because the cosine scores are bounded which leads to uniformity while clustering. The cosine scoring with PIC gives 43% and 54% relative improvements for dev and eval respectively for known N^* over the baseline system. The SSC algorithm performed with PIC provides additional gains. The incorporation of temporal continuity also improves results. We obtain significant relative improvements (60% for both dev and eval) for known N^*. For unknown N^*, we obtain relative improvement of 53% and 59% for dev and eval respectively over the baseline system.

H. Comparison With Other Published Works

Prior works on AMI report results on three domains (Edinburgh, Idiap and Brno) out of the four domains. For comparison, we have also shown SSC-PIC results for these three domains in Table IV. As seen here, the proposed SSC algorithm advances the state-of-the-art results significantly for both the known/unknown N^* condition.

TABLE IV: Comparison of the proposed SSC algorithm with other published works on the AMI dataset (after removing recordings from TNO domain as reported in other works).

System	DER known N*		DER unknown N*	
	Dev.	Eval.	Dev.	Eval.
Semi-sup learning [55]	-	-	17.5	22.0
Incremental learning [56]	-	15.6	-	20.0
GAN clustering [41]	10.2	10.1	11.0	11.3
2-D self attention [57]	-	-	12.2	13.0
Baseline	14.4	16.5	12.9	13.6
SSC-PIC	**4.6**	**6.5**	**5.2**	**5.4**

TABLE V: Comparison of the proposed SSC algorithm based results with other published works on the CH dataset. Here, VB refers to the use of VB-HMM based refinement.

System	DER Known N*	DER unknown N*
x-vec [34] (+VB)	-	12.8 (9.9)
VB modeling [26]	-	9.0
Bootstrap network [40]	6.2	8.6
LSTM scoring [38]	-	7.7
UIS-RNN [18]	-	7.6
Spec. Cluster [54]	-	7.3
Baseline (+VB)	7.0 (5.0)	8.0 (6.4)
SSC-PIC (+VB)	**6.3** (**4.8**)	**7.0** (**5.6**)

A similar comparison with other published results on the CH dataset is shown in Table V. A much larger pool of published results exist on the CH dataset in the recent years. Based on our survey of recent literature, the best reported results use the eigen-gap based spectral clustering [54]. The SSC approach improves over state-of-the-art in CH dataset results as well. However, improvements on AMI dataset shows more significant improvements compared to CH dataset with PIC. One major reason is the difference in the duration of recordings between the datasets. In PIC, the graph is more stable when there are more nodes to build the edge connections. The duration of AMI recordings ranges from 20-60 mins which generates 1000-4000 embeddings. In the CH dataset, the recording duration ranges from 2 -5mins generating 50-400 embeddings. Further, the SSC training also benefits from longer duration as we have more triplets ($> 200,000$) from AMI to train the network as opposed to CH dataset with $< 50,000$ triplets.

The SSC algorithm performance is also shown to be superior to the fully supervised diarization algorithms (for example, the RNN based algorithm [18]) which use a significantly large amount of speaker supervised conversational data. In contrast, the proposed SSC algorithm is purely unsupervised and it is developed without any additional training data over the baseline system. Table IV and V therefore highlight the advantages of self-supervised learning and graph structural clustering used in the SSC algorithm. The SSC outputs at segment level can also be used as initialization for further refinement using frame-level VB-HMM modeling [26]. As seen in Table V, this VB-HMM based refinement step further improves the diarization performance to achieve a DER of 4.8% for known N^* case and 5.6% for unknown N^* case on the CH dataset.

I. Computational Complexity

The training of DNN and the PIC are two stages to be considered for time complexity at recording level. Based on the triplet sampling strategy discussed in Section III-D, the time complexity for training the DNN at each epoch is $\mathcal{O}(N_r^2)$. The time complexity for clustering stage is based on choice of algorithm. The baseline AHC has $\mathcal{O}(N_r^3)$ complexity. The proposed PIC algorithm is more efficient in computation as it calculates incremental path integral in linear time and does not require re-computation of similarity scores. It has time complexity of $\mathcal{O}(N_r^2)$. The overall time complexity of SSC-PIC is $\mathcal{O}(RQN^2)$, where R is the number of DNN epochs and Q is the number of SSC iterations performed. Comparing with the baseline system complexity of $\mathcal{O}(N_r^3)$, the proposed SSC algorithm is more efficient when $N_r > RQ$. This condition is satisfied for recordings that are 30s or more in duration. We performed a compute-time measurement for the baseline (PLDA + AHC) and SSC-PIC system using x-vectors on an Intel CPU server (single core 64-bit). The compute time for the baseline and proposed SSC were $71.6s$ and $43.3s$ respectively for a file of duration 26 minutes from the AMI dev set. This shows that the SSC algorithm achieves an improved DER performance while also reducing the computational complexity.

VI. Conclusion

In this paper, we have proposed an algorithm for self-supervised embedding learning and clustering based on graph-structural path integral. The steps of embedding learning and path integral clustering (PIC) are performed iteratively for the given recording using x-vector features as inputs. The iterative steps validate the hypothesis that improved clustering can be achieved using discriminative representations while self-supervised representations can be learnt with well-separated clusters. The proposed approach, termed as self-supervised clustering (SSC), is applied for speaker diarization task in CH and AMI datasets. The diarization error rates achieved in these tasks are shown to improve the baseline system as well as several other recent state-of-the-art techniques proposed in the field. To the best of our knowledge, the DER results reported in this work constitute the lowest error rates reported till date for diarization on CH and AMI datasets.

Appendix A - Proof of Path Integral

The unnormalized path in Equation 3 is:

$$\begin{aligned} g_{ij} &= \delta_{ij} + \sum_{k=1}^{\infty} \sigma^k \sum_{\gamma \in \Gamma_{ij}^{(k)}} \prod_{s=1}^{k} p_{u_{s-1},u_s} \\ &= \delta_{ij} + \sum_{k=1}^{\infty} \sigma^k [\boldsymbol{P}_{\mathcal{C}_a}^k]_{ij} \\ &= [\boldsymbol{I} + \sum_{k=1}^{\infty} \sigma^k \boldsymbol{P}_{\mathcal{C}_a}^k]_{ij} \\ &= [(\boldsymbol{I} - \sigma \boldsymbol{P}_{\mathcal{C}_a})^{-1}]_{ij} \end{aligned}$$

The normalized path integral $\mathcal{S}_{\mathcal{C}_a}$ (Equation 4) is given as,

$$[\mathcal{S}_{\mathcal{C}_a}]_{ij} = \frac{1}{|\mathcal{C}_a|^2} [(\boldsymbol{I} - \sigma \boldsymbol{P}_{\mathcal{C}_a})^{-1}]_{ij}$$

References

[1] N. Ryant, K. Church, C. Cieri, A. Cristia, J. Du, S. Ganapathy, and M. Liberman, "First DIHARD challenge evaluation plan," *2018, tech. Rep.*, 2018.

[2] N. Ryant, K. Church, C. Cieri, A. Cristia, J. Du, and S. Ganapathy, "The second DIHARD diarization challenge: Dataset, task, and baselines," in *Proc. of Interspeech*, 2019, pp. 978–982.

[3] N. Ryant, P. Singh, V. Krishnamohan, R. Varma, K. Church, C. Cieri, J. Du, S. Ganapathy, and M. Liberman, "The third dihard diarization challenge," *arXiv preprint arXiv:2012.01477*, 2020.

[4] S. E. Tranter and D. A. Reynolds, "An overview of automatic speaker diarization systems," *IEEE Transactions on audio, speech, and language processing*, vol. 14, no. 5, pp. 1557–1565, 2006.

[5] G. Sell and D. Garcia-Romero, "Speaker diarization with plda i-vector scoring and unsupervised calibration," in *IEEE Spoken Language Technology Workshop (SLT)*, 2014, pp. 413–417.

[6] D. Snyder, D. Garcia-Romero, G. Sell, D. Povey, and S. Khudanpur, "X-vectors: Robust DNN embeddings for speaker recognition," in *IEEE ICASSP*, 2018, pp. 5329–5333.

[7] A. Kanagasundaram, S. Sridharan, S. Ganapathy, P. Singh, and C. Fookes, "A study of x-vector based speaker recognition on short utterances," in *Proc of Interspeech*, 2019, pp. 2943–2947.

[8] X. Anguera, S. Bozonnet, N. Evans, C. Fredouille, G. Friedland, and O. Vinyals, "Speaker diarization: A review of recent research," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 20, no. 2, pp. 356–370, 2012.

[9] X. Anguera, C. Wooters, and J. M. Pardo, "Robust speaker diarization for meetings: ICSI RT06s meetings evaluation system," in *International Workshop on Machine Learning for Multimodal Interaction*. Springer, 2006, pp. 346–358.

[10] W. H. Day and H. Edelsbrunner, "Efficient algorithms for agglomerative hierarchical clustering methods," *Journal of classification*, vol. 1, no. 1, pp. 7–24, 1984.

[11] M. Senoussaoui, P. Kenny, T. Stafylakis, and P. Dumouchel, "A study of the cosine distance-based mean shift for telephone speech diarization," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 22, no. 1, pp. 217–227, 2014.

[12] D. Garcia-Romero and C. Y. Espy-Wilson, "Analysis of i-vector length normalization in speaker recognition systems," in *Proc. of Interspeech*, 2011, pp. 249–252.

[13] W. Zhu and J. Pelecanos, "Online speaker diarization using adapted i-vector transforms," in *IEEE ICASSP*, 2016, pp. 5045–5049.

[14] S. Ramoji, P. Krishnan, and S. Ganapathy, "NPLDA: A Deep Neural PLDA Model for Speaker Verification," in *Proc. Odyssey 2020 The Speaker and Language Recognition Workshop*, 2020, pp. 202–209.

[15] I. Viñals, A. Ortega, J. A. V. López, A. Miguel, and E. Lleida, "Domain adaptation of plda models in broadcast diarization by means of unsupervised speaker clustering," in *Proc. of Interspeech*, 2017, pp. 2829–2833.

[16] H. Ning, M. Liu, H. Tang, and T. S. Huang, "A spectral clustering approach to speaker diarization," in *Proc. of Interspeech and ICSLP*, 2006, pp. 2178–2181.

[17] D. Vijayasenan, F. Valente, and H. Bourlard, "An information theoretic approach to speaker diarization of meeting data," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 17, no. 7, pp. 1382–1393, 2009.

[18] A. Zhang, Q. Wang, Z. Zhu, J. Paisley, and C. Wang, "Fully supervised speaker diarization," in *IEEE ICASSP*, 2019, pp. 6301–6305.

[19] Y. Fujita, N. Kanda, S. Horiguchi, K. Nagamatsu, and S. Watanabe, "End-to-End Neural Speaker Diarization with Permutation-Free Objectives," in *Proc. of Interspeech*, 2019, pp. 4300–4304.

[20] D. Hendrycks, M. Mazeika, S. Kadavath, and D. Song, "Using self-supervised learning can improve model robustness and uncertainty," in *Advances in Neural Information Processing Systems*, 2019, pp. 15 663–15 674.

[21] S. Pascual, M. Ravanelli, J. Serrà, A. Bonafonte, and Y. Bengio, "Learning problem-agnostic speech representations from multiple self-supervised tasks," in *Proc. of Interspeech*, 2019, pp. 161–165.

[22] W. Zhang, D. Zhao, and X. Wang, "Agglomerative clustering via maximum incremental path integral," *Pattern Recognition*, vol. 46, no. 11, pp. 3056–3065, 2013.

[23] P. Singh and S. Ganapathy, "Deep Self-Supervised Hierarchical Clustering for Speaker Diarization," in *Proc. of Interspeech*, 2020, pp. 294–298.

[24] M. Przybocki and A. Martin, "2000 NIST Speaker Recognition Evaluation," https://catalog.ldc.upenn.edu/LDC2001S97.

[25] I. McCowan, J. Carletta, W. Kraaij, S. Ashby, S. Bourban, M. Flynn, M. Guillemot, T. Hain, J. Kadlec, V. Karaiskos *et al.*, "The AMI meeting corpus," in *Proceedings of Measuring Behavior 2005, 5th International Conference on Methods and Techniques in Behavioral Research*. Noldus Information Technology, 2005, pp. 137–140.
[26] M. Diez, L. Burget, and P. Matejka, "Speaker diarization based on bayesian hmm with eigenvoice priors." in *Odyssey*, 2018, pp. 147–154.
[27] P. Singh, H. Vardhan, S. Ganapathy, and A. Kanagasundaram, "Leap diarization system for the second dihard challenge," in *Proc. of Interspeech): Crossroads of Speech and Language*. International Speech Communication Association, 2019, pp. 983–987.
[28] D. A. Reynolds and P. Torres-Carrasquillo, "Approaches and applications of audio diarization," in *IEEE ICASSP*, 2005, pp. 953–956.
[29] C. Wooters and M. Huijbregts, "The ICSI RT07s speaker diarization system," in *Multimodal Technologies for Perception of Humans*. Springer, 2007, pp. 509–519.
[30] N. Dehak, P. J. Kenny, R. Dehak, P. Dumouchel, and P. Ouellet, "Front-end factor analysis for speaker verification," *IEEE Transactions on Audio, Speech, and Language Processing*, vol. 19, no. 4, pp. 788–798, 2010.
[31] J. Silovsky and J. Prazak, "Speaker diarization of broadcast streams using two-stage clustering based on i-vectors and cosine distance scoring," in *IEEE ICASSP*, 2012, pp. 4193–4196.
[32] S. Ioffe, "Probabilistic linear discriminant analysis," in *European Conference on Computer Vision*. Springer, 2006, pp. 531–542.
[33] Y. Lei, L. Burget, L. Ferrer, M. Graciarena, and N. Scheffer, "Towards noise-robust speaker recognition using probabilistic linear discriminant analysis," in *IEEE ICASSP*, 2012, pp. 4253–4256.
[34] D. Garcia-Romero, D. Snyder, G. Sell, D. Povey, and A. McCree, "Speaker diarization using deep neural network embeddings," in *IEEE ICASSP*, 2017, pp. 4930–4934.
[35] Q. Wang, C. Downey, L. Wan, P. A. Mansfield, and I. L. Moreno, "Speaker diarization with LSTM," in *IEEE ICASSP*, 2018, pp. 5239–5243.
[36] P. Cyrta, T. Trzciński, and W. Stokowiec, "Speaker diarization using deep recurrent convolutional neural networks for speaker embeddings," in *International Conference on Information Systems Architecture and Technology*. Springer, 2017, pp. 107–117.
[37] S. Shum, N. Dehak, E. Chuangsuwanich, D. Reynolds, and J. Glass, "Exploiting intra-conversation variability for speaker diarization," in *Proc. of Interspeech*, 2011, pp. 945–948.
[38] Q. Lin, R. Yin, M. Li, H. Bredin, and C. Barras, "LSTM Based Similarity Measurement with Spectral Clustering for Speaker Diarization," in *Proc. of Interspeech*, 2019, pp. 366–370.
[39] S. Horiguchi, Y. Fujita, S. Watanabe, Y. Xue, and K. Nagamatsu, "End-to-End Speaker Diarization for an Unknown Number of Speakers with Encoder-Decoder Based Attractors," in *Proc. of Interspeech*, 2020, pp. 269–273.
[40] Q. Li, F. L. Kreyssig, C. Zhang, and P. C. Woodland, "Discriminative neural clustering for speaker diarisation," *arXiv preprint arXiv:1910.09703*, 2019.
[41] M. Pal, M. Kumar, R. Peri, T. J. Park, S. H. Kim, C. Lord, S. Bishop, and S. Narayanan, "Speaker diarization using latent space clustering in generative adversarial network," in *IEEE ICASSP*, 2020, pp. 6504–6508.
[42] M. Diez, L. Burget, F. Landini, and J. Černockỳ, "Analysis of speaker diarization based on bayesian HMM with eigenvoice priors," *IEEE/ACM Transactions on Audio, Speech, and Language Processing*, vol. 28, pp. 355–368, 2019.
[43] B. Yang, X. Fu, N. D. Sidiropoulos, and M. Hong, "Towards k-means-friendly spaces: Simultaneous deep learning and clustering," in *international conference on machine learning*, 2017, pp. 3861–3870.
[44] U. Shaham, K. Stanton, H. Li, R. Basri, B. Nadler, and Y. Kluger, "Spectralnet: Spectral clustering using deep neural networks," in *International Conference on Learning Representations*, 2018.
[45] J. Yang, D. Parikh, and D. Batra, "Joint unsupervised learning of deep representations and image clusters," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 5147–5156.
[46] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning – Data Mining, Inference, and Prediction*. New York: Springer, 2009.
[47] M. Pavan and M. Pelillo, "Dominant sets and pairwise clustering," *IEEE transactions on pattern analysis and machine intelligence*, vol. 29, no. 1, pp. 167–172, 2006.
[48] Y. Tang, X. Wu, and W. Bu, "Saliency detection based on graph-structural agglomerative clustering," in *Proceedings of the 23rd ACM international conference on Multimedia*, 2015, pp. 1083–1086.
[49] D. Povey, A. Ghoshal, G. Boulianne, L. Burget, O. Glembek, N. Goel, M. Hannemann, P. Motlicek, Y. Qian, P. Schwarz *et al.*, "The Kaldi speech recognition toolkit," in *IEEE workshop on automatic speech recognition and understanding*. IEEE Signal Processing Society, 2011.
[50] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
[51] A. Nagrani, J. S. Chung, and A. Zisserman, "Voxceleb: A large-scale speaker identification dataset," *Proc. of Interspeech*, pp. 2616–2620, 2017.
[52] A. Nagrani, J. S. Chung, W. Xie, and A. Zisserman, "Voxceleb: Large-scale speaker verification in the wild," *Computer Speech & Language*, vol. 60, p. 101027, 2020.
[53] L. V. D. Maaten and G. Hinton, "Visualizing data using t-sne," *Journal of machine learning research*, vol. 9, no. Nov, pp. 2579–2605, 2008.
[54] T. J. Park, K. J. Han, M. Kumar, and S. Narayanan, "Auto-tuning spectral clustering for speaker diarization using normalized maximum eigengap," *IEEE Signal Processing Letters*, vol. 27, pp. 381–385, 2019.
[55] M. Pal, M. Kumar, R. Peri, and S. Narayanan, "A study of semi-supervised speaker diarization system using gan mixture model," *arXiv preprint arXiv:1910.11416*, 2019.
[56] N. Dawalatabad, S. Madikeri, C. C. Sekhar, and H. A. Murthy, "Incremental transfer learning in two-pass information bottleneck based speaker diarization system for meetings," in *IEEE ICASSP*, 2019, pp. 6291–6295.
[57] G. Sun, C. Zhang, and P. C. Woodland, "Speaker diarisation using 2D self-attentive combination of embeddings," in *IEEE ICASSP*, 2019, pp. 5801–5805.

Prachi Singh is a graduate student at Learning and Extraction of Acoustic Patterns (LEAP) lab, Electrical Engineering, Indian Institute of Science, Bangalore. In the past, she has worked in Fiat Chrysler Automobiles, Chennai, India from 2015 to 2017. She obtained her Bachelor of Technology in Electronics and Telecommunication from College of Engineering, Pune in 2015. Her research interests include speaker diarization, speaker verification, self-supervised learning and graph clustering. She is a student member of IEEE and ISCA.

Sriram Ganapathy is a faculty member at the Electrical Engineering, Indian Institute of Science, Bangalore, where he heads the activities of the learning and extraction of acoustic patterns (LEAP)lab. Prior to joining the Indian Institute of Science,he was a research staff member at the IBM Watson Research Center, Yorktown Heights. He received his Doctor of Philosophy from the Center for Language and Speech Processing, Johns Hopkins University. He obtained his Bachelor of Technology from College of Engineering, Trivandrum, India and Master of Engineering from the Indian Institute of Science, Bangalore. He has also worked as a Research Assistant in Idiap Research Institute, Switzerland from 2006 to 2008. At the LEAP lab, his research interests include signal processing, machine learning methodologies for speech and speaker recognition and auditory neuroscience. He is a subject editor for the Speech Communications journal, member of ISCA and senior member of IEEE.